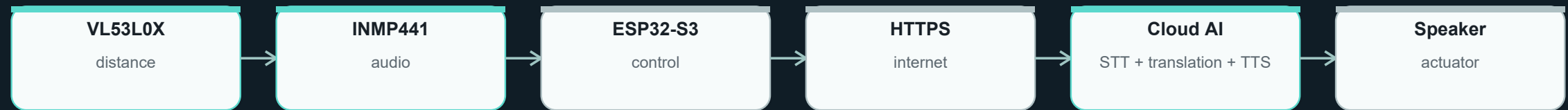


Understand Any Language (UAL)

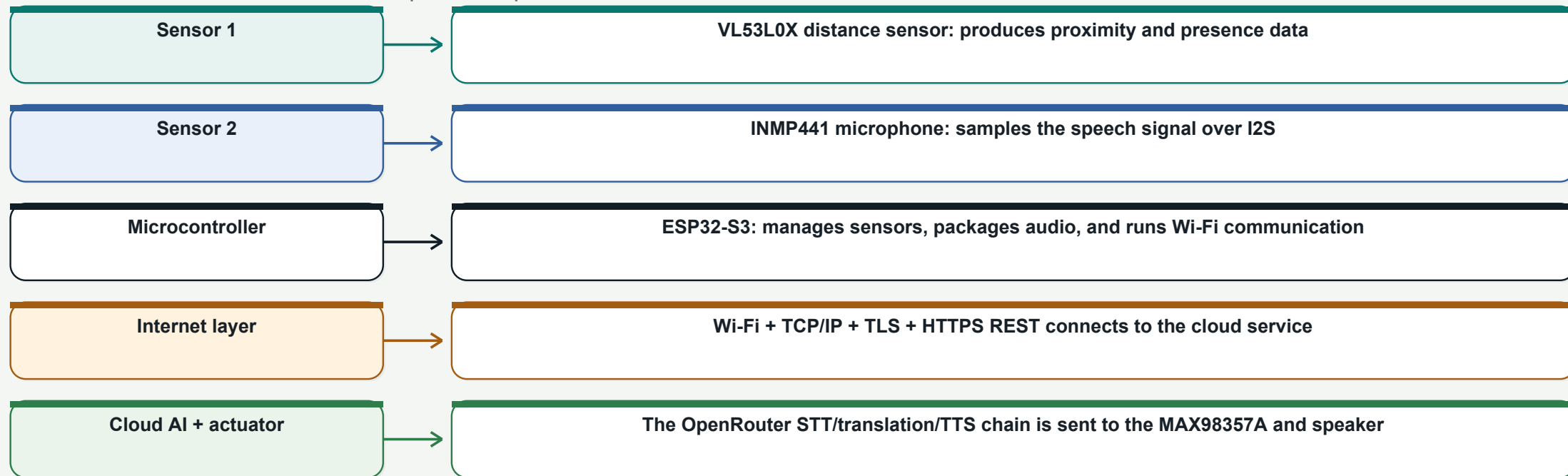
Understand any language in real life. This two-sensor embedded device senses the environment, captures speech, sends it through a cloud AI pipeline, and plays the result through the speaker.



SYSTEM MAP

A closed data loop from sensor to actuator

The prototype works as an IoT node that collects environmental data, processes it over the internet, and turns the result into spoken output.



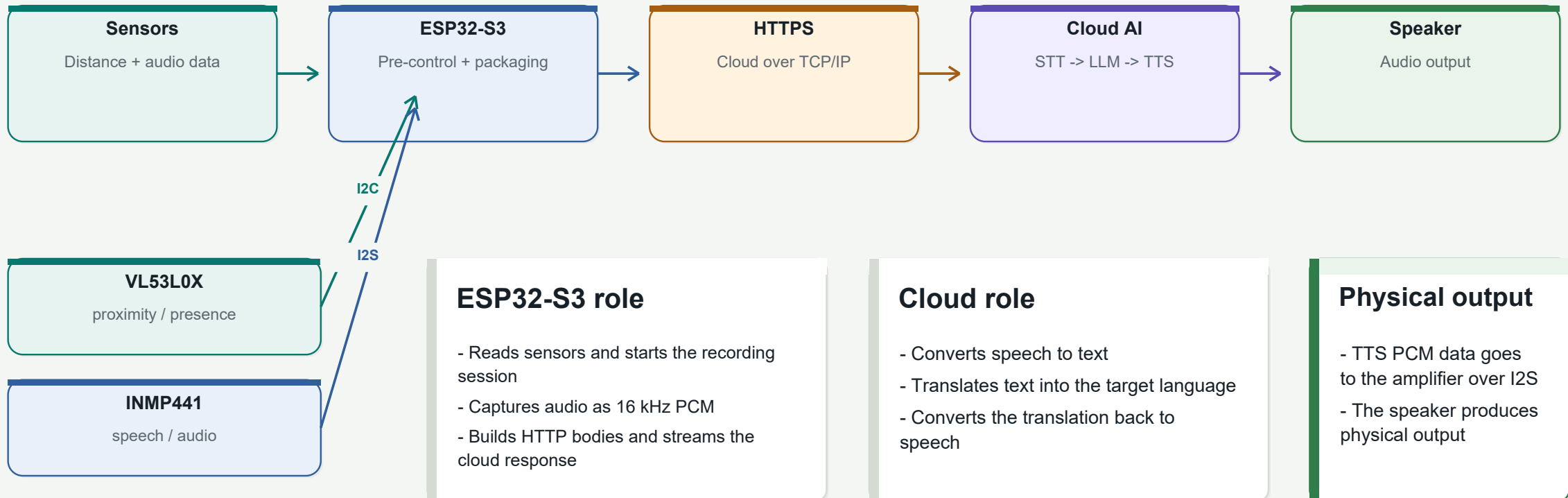
Data loop

Distance and audio data from the physical environment are controlled on the ESP32-S3, processed in the cloud AI chain, and returned to the physical world through the speaker.

MAIN ARCHITECTURE

Data moves from the physical world to the cloud and back

Each block is a link in the chain: sensing, control, internet transport, cloud AI processing, and physical output.



SENSOR LAYER

The device senses the environment through distance and audio

The distance sensor starts the recording session; the microphone moves speech data into ESP32-S3 memory in regular chunks.

VL53L0X distance sensor

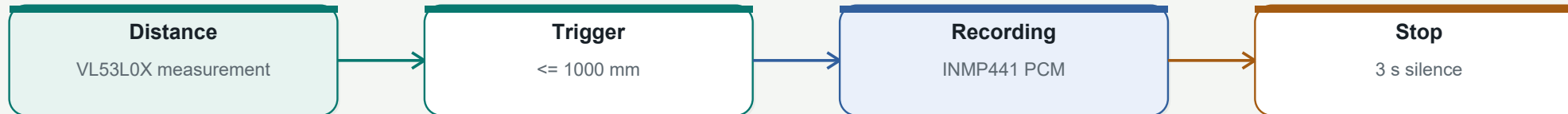
- Read over I2C every 150 ms
- Presence triggers at 1000 mm or less
- Presence clears above 1200 mm
- 3.5 s retrigger delay

INMP441 microphone sensor

- Reads 32-bit samples over I2S RX
- Samples are downconverted to 16-bit PCM
- Processed in 320-sample chunks
- RMS and peak thresholds determine speech presence

Recording session rules

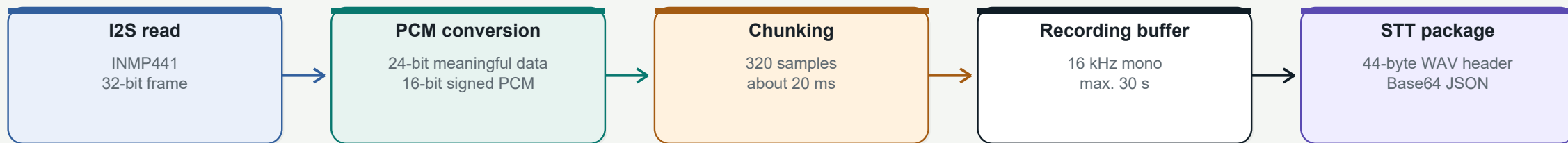
- Starts by automatic trigger or web/serial command
- Recording stops after 3 seconds of silence
- 30-second maximum recording window
- Cloud processing does not start if no speech is detected



AUDIO DATA

The microphone does not create a file; ESP32-S3 turns the sample stream into a PCM recording

Speech first arrives as raw I2S samples, then is written into a 16 kHz mono PCM buffer and converted into a WAV/Base64 body for the cloud.



Sampling format

- Sample rate: 16 kHz
- Channel: mono
- Sample type: signed PCM16
- 30 s recording produces about 960 KB raw PCM

Speech detection

- Chunk average is measured and DC offset is removed
- RMS threshold: 900
- Peak threshold: 2500
- Speech is accepted when both thresholds pass

Cloud preparation

- Raw PCM is not sent directly to STT
- A WAV header is prepended
- WAV data is carried as base64 inside JSON
- The large body is written in chunks

NETWORK LAYER

ESP32-S3 sends data to OpenRouter with HTTPS REST requests

The network flow starts with Wi-Fi, passes through DNS, TCP/IP, and TLS, then becomes HTTP POST requests.



STT request

- Endpoint: /api/v1/audio/transcriptions
- Body: model + input_audio JSON
- Audio: base64 data with WAV header
- Response: transcript text

Translation request

- Endpoint: /api/v1/chat/completions
- Body: system + user messages
- Temperature 0 for steadier translation
- Response: target-language text

TTS request

- Endpoint: /api/v1/audio/speech
- Body: input, voice, speed, response_format
- response_format: pcm
- Response: Content-Length or chunked PCM stream

CLOUD LAYER

Three separate AI tasks run in sequence on OpenRouter

ESP32-S3 does not run the models; it sends audio and text to the cloud, reads cloud responses, and passes them to the next step.

1. STT - Speech to Text

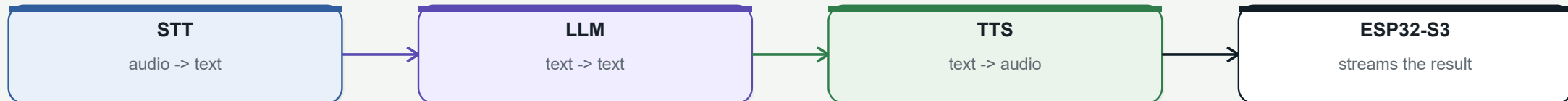
- Model: openai/whisper-large-v3
- Task: convert speech to text
- The Whisper family uses transformer-based speech recognition
- Output: transcript text

2. LLM translation

- Model: mistralai/mistral-small-2603
- Task: translate the transcript into the target language
- Generates text with decoder-only LLM behavior
- Output: translated text only

3. TTS - Text to Speech

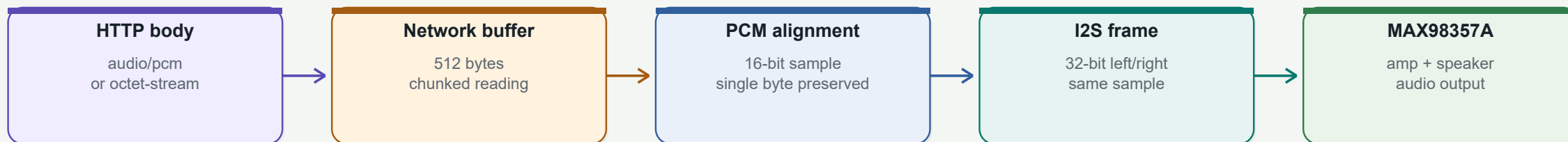
- Model: openai/gpt-4o-mini-tts-2025-12-15
- Task: turn translated text into speech
- Neural TTS: produces a speech waveform from text
- Voice: alloy, speed: 1.0
- Output: raw PCM audio stream



ACTUATOR LAYER

The TTS response is not downloaded as a file; the PCM stream is played in chunks

Firmware aligns raw PCM bytes from the HTTP body, converts them to I2S output, and sends them through the MAX98357A amplifier to the speaker.



Stream reading

- If the response is chunked, each HTTP chunk is read separately
- If Content-Length exists, remaining byte count is tracked
- Large audio is not fully loaded into RAM

PCM playback

- PCM16 samples are kept 2-byte aligned
- Playback is multiplied by gain
- Overflow is clipped to int16 range
- Sample is expanded to a 32-bit I2S frame

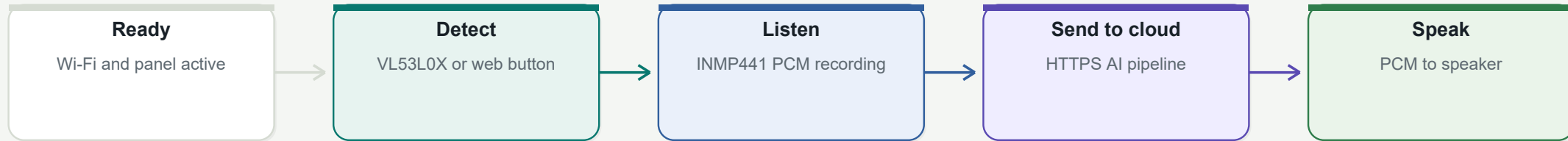
Speaker setup

- Default TTS sample rate is 24 kHz
- The I2S driver restarts if needed
- Mono audio is copied to left and right channels
- MAX98357A converts the digital I2S signal to analog sound

DEVICE BEHAVIOR

Firmware behaves like a state machine

The device listens to the web panel, tracks sensors, manages recording state, and returns to ready state after cloud processing.



Automatic trigger

- Distance sensor is read at regular intervals
- A new recording starts when the user moves below the threshold
- Manual recording is also possible from the web panel

Recording management

- Speech is stored as 16 kHz mono PCM
- Recording ends when silence is detected
- The maximum recording window prevents memory overflow

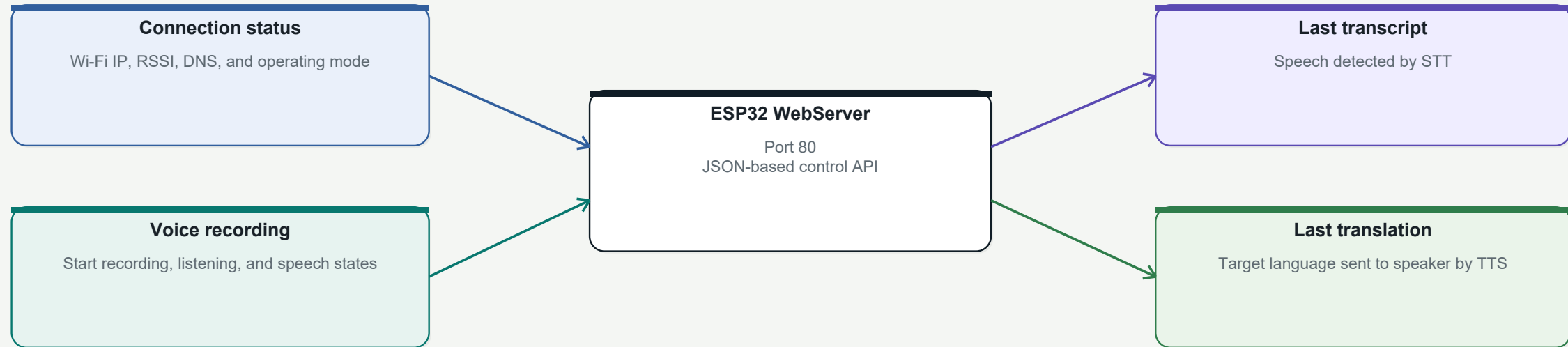
Feedback

- Status is shown on the web panel
- Last text and translation are visible in the panel
- Speaker output is the system's physical response

HUMAN INTERFACE

The web panel makes the device's IoT behavior visible

The browser interface shows connection status, recording phase, transcript, and translation result live.



During use

- First, IP and sensor readiness are shown
- Then recording starts and the pipeline is tracked

Observability

- The system does not look like a black box
- Data, status, and output are monitored on one screen

Settings management

- Wi-Fi, language, model, and audio settings are saved
- The status API periodically updates the web panel

CLOSING

The project is completed as an IoT pipeline

The focus is no longer the number of parts, but the closed loop between physical data, network, cloud AI, and physical output.

Strengths

- Sensor, network, cloud AI, and actuator layers are clearly separated
- The cloud AI chain is live and observable
- The web panel makes diagnostics and monitoring easier

Limitations

- Requires an internet connection and OpenRouter access
- Ambient noise can affect STT success
- API key and network access must be controlled

Improvements

- Authenticated web panel on the device side
- Move TLS certificate verification to production level
- Noise reduction and better microphone calibration

Closing message

ESP32-S3 works as an end-to-end IoT prototype that sends real-world data from two sensors to cloud AI over HTTPS and plays the result through a speaker actuator.